Q1)

1. Design:

```
module Dynamic_Arr();

    int dyn_arr1[], dyn_arr2[] = '{9, 1, 8, 3, 4, 4};


    initial begin
        dyn_arr1 = new[6];
        foreach (dyn_arr1[i]) begin
            dyn_arr1[i] = i;
        end

        $display("dyn_arr1: %p", dyn_arr1);
        dyn_arr1.delete();

        dyn_arr2.reverse();
        $display("dyn_arr2 Reversed: %p", dyn_arr2);

        dyn_arr2.sort();
        $display("dyn_arr2 Sorted: %p", dyn_arr2);

        dyn_arr2.rsort();
        $display("dyn_arr2 Reverse Sorted: %p", dyn_arr2);

        dyn_arr2.shuffle();
        $display("dyn_arr2 Shuffled: %p", dyn_arr2);
    end


endmodule
```

2. Questasim Output:

```
# (vsim-4027) Logging is not supported for Dynamic Array item.
VSIM 3> run -all
# dyn_arr1: '{0, 1, 2, 3, 4, 5}
# dyn_arr2 Reversed: '{4, 4, 3, 8, 1, 9}
# dyn_arr2 Sorted: '{1, 3, 4, 4, 8, 9}
# dyn_arr2 Reverse Sorted: '{9, 8, 4, 4, 3, 1}
# dyn_arr2 Shuffled: '{8, 1, 4, 3, 9, 4}

VSIM 4>
```

Q2)

1. Design:

```
1  ////////////////////////////////////////////////////////////////////////////////
2  // Author: Kareem Waseem
3  // Course: Digital Verification using SV & UVM
4  //
5  // Description: Counter Design
6  //
7  ////////////////////////////////////////////////////////////////////////////////
8  module counter (clk ,rst_n, load_n, up_down, ce, data_load, count_out, max_count, zero);
9  parameter WIDTH = 4;                //(Valid values: 4, 6, 8, default: 4)
10 input clk;
11 input rst_n;                        //(active low sync rst)
12 input load_n;                       //(active low load)
13 input up_down;                      //high then increment counter, else decrement
14 input ce;                           //(clock enable signal
15 input [WIDTH-1:0] data_load;         //(load data to count_out output when the load_n signal is asserted)
16 output reg [WIDTH-1:0] count_out;
17 output max_count;                   //counter reaches the maximum value, this signal is high, else low)
18 output zero;                        //counter reaches the minimum value, this signal is high, else low)
19
20 always @(posedge clk) begin
21     if (~rst_n)                     //Modification: changed comparison "!" to bitwise NOT "~"
22         count_out <= 0;
23     else if (~load_n)               //Modification: changed comparison "!" to bitwise NOT "~"
24         count_out <= data_load;
25     else if (ce)begin               //Fixed BUg: added Begin and End statments
26         if (up_down)
27             count_out <= count_out + 1;
28         else
29             count_out <= count_out - 1;
30     end
31 end
32
33 assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
34 assign zero = (count_out == 0)? 1:0;
35
36 endmodule
```

2. Verification Plan:

Label	Design Requirment Description	Stimulus Generation	Funcational Coverage	Funcational Checking
COUNTER_1	When the reset is asserted, the output counter value should be low	Directed at the start of the sim, then randomized with constraint that drive the reset to be off most of the simulation time.		A checker in the testbench to make sure the output is correct
COUNTER_2	When the load is asserted, the output count_out should take the value of the load_data input	Randomization under constraints on the load signal to be off 70% of the time		A checker in the testbench to make sure the output is correct
COUNTER_3	When the enable is not asserted, the output counter value should be unchanged	Randomization under constraints on the load signal to be off 70% of the time		A checker in the testbench to make sure the output is correct
COUNTER_4	when the up_down the value of counter_out should increment	Directed at the start of the simulation		A checker in the testbench to make sure the output is correct
COUNTER_5	when the up_down has a value of low the value of counter_out should decrement	Directed at the start of the simulation		A checker in the testbench to make sure the output is correct
COUNTER_6	When the count_up has MAX value max_count should be high	Directed at the start of the simulation		A checker in the testbench to make sure the output is correct
COUNTER_7	When the count_up has Min value zero should be high	Directed at the start of the simulation		A checker in the testbench to make sure the output is correct

3. Testbench Code

```
import counter_pkg::*;

module counter_tb();

    logic clk;
    logic rst_n;                        //(active low sync rst)
    logic load_n;                       //(active low load)
    logic up_down;                      //high then increment counter, else decrement
    logic ce;                           //(clock enable signal
    logic [WIDTH-1:0] data_load;        //(load data to count_out output when the load_n signal is asserted)
    logic [WIDTH-1:0] count_out;
    logic [WIDTH-1:0] count_out_expected;
    logic max_count;                   //counter reaches the maximum value, this signal is high, else low)
    logic zero;                        //counter reaches the minimum value, this signal is high, else low)

    counter #(WIDTH) DUT (.*);
    N_bit_up_down_Counter #(WIDTH) Golden(.clk(clk),
                                          .reset(rst_n),
                                          .en_load(load_n),
                                          .clk_en(ce),
                                          .load(data_load),
                                          .up_ndown(up_down),
                                          .cnt(count_out_expected));

    Counter_inputs inputs = new;

    int error_count, correct_count;

    initial begin
        clk = 0;
        forever
            #1 clk = ~clk;
    end

    task check_result();
        @(negedge clk);
        $display("Expected:%d | DUT:%d |  UP_DOWN:%d | CLK_EN:%d | Load_n:%d | Load Data:%d", count_out_expected, count_out
                                                                                              , up_down, ce, load_n, data_load);
        if(count_out !== count_out_expected) begin
            error_count++;
            // $stop;
        end
        else begin
            correct_count++;
        end
    endtask

    initial begin
        assert(inputs.randomize());
        //COUNTER_1
        rst_n = 0;
        load_n = inputs.load_n;
        up_down = inputs.up_down;
        ce = inputs.ce;
        data_load = inputs.data_load;

        check_result();

        //COUNTER_2, 3, 4, 5
        for(int i = 0; i < 100; i++)begin
            assert(inputs.randomize());
            rst_n = inputs.rst_n;
            load_n = inputs.load_n;
            up_down = inputs.up_down;
            ce = inputs.ce;
            data_load = inputs.data_load;

            check_result();
        end

```

```
        //COUNTER_6
        rst_n = 1; load_n = 0; ce = 1; data_load = 4'b1111;
        @(negedge clk)
        if(max_count != 1)begin
            $display("Error In Max_Count: Expected:1 | DUT:
", max_count);error_count++;
            // $stop;
        end
        else begin
            correct_count++;
        end

        //COUNTER_7
        rst_n = 1; load_n = 0; ce = 1; data_load = 0;
        @(negedge clk)
        if(zero != 1)begin
            $display("Error In zero: Expected:1 | DUT:
", zero);   error_count++;
            // $stop;
        end
        else begin
            correct_count++;
        end

        $display("Total Tests:%d | Correct Tests:%d | Error Test:%d", correct_count + error_count, correct_count, error_count);
        $stop;

    end

endmodule
```

Package Code:

```
package counter_pkg;

    parameter WIDTH = 4;

    class Counter_inputs;

        rand logic rst_n;                       //(active low sync rst)
        rand logic load_n;                      //(active low load)
        rand logic up_down;                     //high then increment counter, else decrement
        rand logic ce;                          //(clock enable signal
        rand logic [WIDTH-1:0] data_load;       //(load data to count_out output when the load_n signal is asserted)

        constraint c_rst_n{
            rst_n dist{0:/10, 1:/90};
        }

        constraint c_load_n{
            load_n dist{0:/30, 1:/70};
        }

        constraint c_ce{
            ce dist{0:/30, 1:/70};
        }

        function new(logic rst_n = 1, logic load_n = 1, logic up_down = 1, logic ce = 0, logic [WIDTH-1:0] data_load = 0);

            this.rst_n = rst_n;
            this.load_n = load_n;
            this.up_down = up_down;
            this.ce = ce;
            this.data_load = data_load;

        endfunction
```

Golden Model:

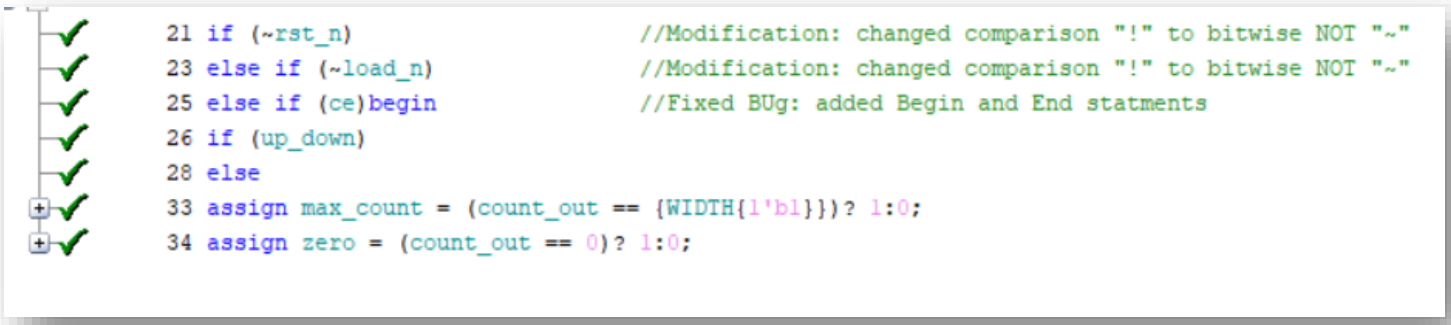
```
module N_bit_up_down_Counter(clk, reset, en_load, clk_en, load, up_ndown, cnt);

parameter WIDTH = 3;
input clk, reset, en_load, up_ndown, clk_en;
input [WIDTH-1: 0] load;
output reg [WIDTH-1: 0] cnt;

always @(posedge clk) begin

    if (~reset)
        cnt <= 0;
    else if(~en_load)
        cnt <= load;
    else if(clk_en)begin
        if(up_ndown)
            cnt <= cnt + 1;
        else
            cnt <= cnt - 1;
    end


end


endmodule
```

4. Do File:

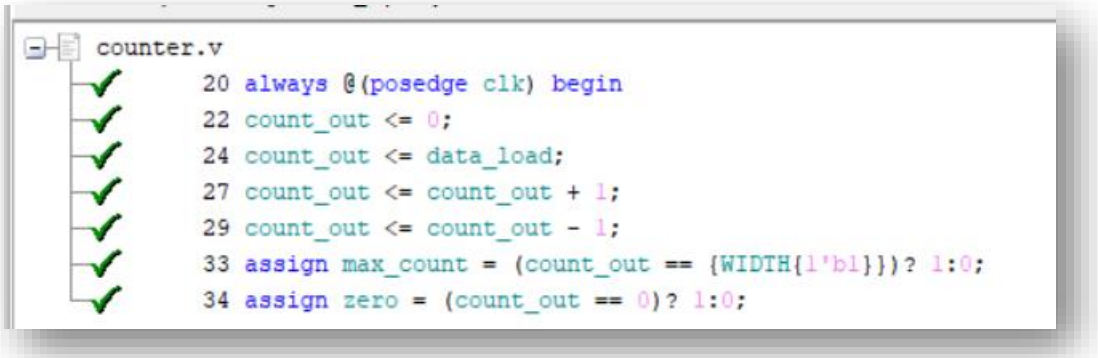
```
vlib work
vlog counter.v counter_golden.v counter_tb.sv counter_pkg.sv +cover -covercells
vsim -voptargs=+acc work.counter_tb -cover
do wave.do
coverage save counter_tb.ucdb -onexit -du work.counter
run -all
quit -sim
vcover report counter_tb.ucdb -details -annotate -all -output coverage_rpt.txt
```

5. Code Coverage:

Branch Coverage:

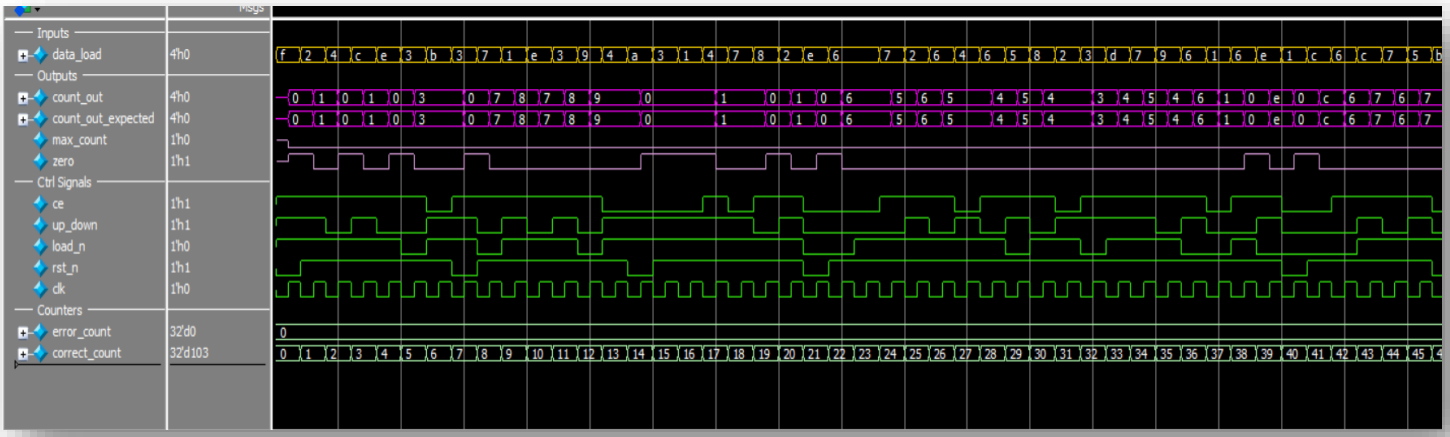


Statement Coverage:



Toggle Coverage:

6. Questasim Waveform



Note: this is the waveform of half the test. I settled with this half for readability

7. Coverage Report:

Branch Coverage:

```
================================================================================
=== Instance: /\counter_tb#DUT
=== Design Unit: work.counter
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        10        10         0   100.00%

================================Branch Details================================

Branch Coverage for instance /\counter_tb#DUT

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File counter.v
------------------------------------IF Branch------------------------------------
    21                                       103     Count coming in to IF
    21              1                         13         if (~rst_n)                  //Modification: changed comparison "!" to bitwise NOT "~"

    23              1                         31         else if (~load_n)            //Modification: changed comparison "!" to bitwise NOT "~"

    25              1                         43         else if (ce)begin            //Fixed BUg: added Begin and End statments

                                              16     All False Count
Branch totals: 4 hits of 4 branches = 100.00%

------------------------------------IF Branch------------------------------------
    26                                        43     Count coming in to IF
    26              1                         17             if (up_down)

    28              1                         26             else

Branch totals: 2 hits of 2 branches = 100.00%
```

Statement Coverage:

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       7         7         0   100.00%

================================Statement Details================================

Statement Coverage for instance /\counter_tb#DUT  --
```

Toggle & Total Coverage:

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         30        30         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /\counter_tb#DUT  --

                                           Node      1H->0L      0L->1H  "Coverage"
                                           ----------------------------------------
                                             ce           1           1      100.00
                                            clk           1           1      100.00
                                  count_out[3-0]          1           1      100.00
                                  data_load[0-3]          1           1      100.00
                                           load_n         1           1      100.00
                                        max_count         1           1      100.00
                                            rst_n         1           1      100.00
                                          up_down         1           1      100.00
                                             zero         1           1      100.00

Total Node Count     =         15
Toggled Node Count   =         15
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (30 of 30 bins)


Total Coverage By Instance (filtered view): 100.00%
```

Q3)

1. Design

```
module ALSU(A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst, direction, leds, out);
parameter INPUT_PRIORITY = "A";
parameter FULL_ADDER = "ON";
input clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
input [2:0] opcode;
input signed [2:0] A, B;
output reg [15:0] leds;
output reg signed [5:0] out;

reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
reg cin_reg;                          //Fixed Bug: CIN supposed to be 1 bit unsigned not 2 bits signed
reg [2:0] opcode_reg;
reg signed [2:0] A_reg, B_reg;

wire invalid_red_op, invalid_opcode, invalid;

//Invalid handling
assign invalid_red_op = (red_op_A_reg | red_op_B_reg) & (opcode_reg[1] | opcode_reg[2]);
assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
assign invalid = invalid_red_op | invalid_opcode;

//Registering input signals
always @(posedge clk or posedge rst) begin
  if(rst) begin
     cin_reg <= 0;
     red_op_B_reg <= 0;
     red_op_A_reg <= 0;
     bypass_B_reg <= 0;
     bypass_A_reg <= 0;
     direction_reg <= 0;
     serial_in_reg <= 0;
     opcode_reg <= 0;
     A_reg <= 0;
     B_reg <= 0;
  end else begin
     cin_reg <= cin;
     red_op_B_reg <= red_op_B;
     red_op_A_reg <= red_op_A;
     bypass_B_reg <= bypass_B;
     bypass_A_reg <= bypass_A;
     direction_reg <= direction;
     serial_in_reg <= serial_in;
     opcode_reg <= opcode;
     A_reg <= A;
     B_reg <= B;
  end
end

//leds output blinking
always @(posedge clk or posedge rst) begin
  if(rst) begin
     leds <= 0;
  end else begin
      if (invalid)
        leds <= ~leds;
      else
        leds <= 0;
  end
end
```

```
//ALSU output processing
always @(posedge clk or posedge rst) begin
  if(rst) begin
    out <= 0;
  end
  else begin
    if (bypass_A_reg && bypass_B_reg)
      out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
    else if (bypass_A_reg)
      out <= A_reg;
    else if (bypass_B_reg)
      out <= B_reg;
    else if (invalid)
       out <= 0;
    else begin
        case (opcode_reg)                          //Fixed Bug: changed incorrect checking opcode to opcode_reg
          3'h0: begin
            if (red_op_A_reg && red_op_B_reg)
              out <= (INPUT_PRIORITY == "A")? |A_reg: |B_reg;
            else if (red_op_A_reg)
              out <= |A_reg;
            else if (red_op_B_reg)
              out <= |B_reg;
            else
              out <= A_reg | B_reg;
          end
          3'h1: begin
            if (red_op_A_reg && red_op_B_reg)
              out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg;
            else if (red_op_A_reg)
              out <= ^A_reg;
            else if (red_op_B_reg)
              out <= ^B_reg;
            else
              out <= A_reg ^ B_reg;
          end
          3'h2: out <= (FULL_ADDER == "ON")? A_reg + B_reg + cin_reg : A_reg + B_reg ; //Fixed Bug: added FULL_ADDER checker
          3'h3: out <= A_reg * B_reg;
          3'h4: begin
            if (direction_reg)
              out <= {out[4:0], serial_in_reg};
            else
              out <= {serial_in_reg, out[5:1]};
          end
          3'h5: begin
            if (direction_reg)
              out <= {out[4:0], out[5]};
            else
              out <= {out[0], out[5:1]};
          end
          default: out <= 0;
        endcase
    end
  end
end

endmodule
```

2. Do file:

```
vlib work
vlog ALSU.v ALSU_Golden_2.v D_FF.v ALSU_tb.sv ALSU_pkg.sv +cover -covercells
vsim -voptargs=+acc work.ALSU_tb -cover
do wave.do
coverage save ALSU_tb.ucdb -onexit -du work.ALSU
run -all
coverage exclude -src ALSU.v -line 111 -code s
coverage exclude -src ALSU.v -line 111 -code b
quit -sim
vcover report ALSU_tb.ucdb -details -annotate -all -output coverage_rpt.txt
```

3. Verification Plan

Label	Design Requirment Description	Stimulus Generation	Funcational Coverage	Funcational Checking
ALSU_1	When the reset is asserted, the outputs should Directed at the start of the be low	Directed at the start of the sim, then randomized with constraint that drive the reset to be off most of the simulation time.		A checker in the testbench to make sure the output is correct
ALSU_2	Incase of invalid cases do not occur, when opcode is add, then out should perform the addition on ports A and B taking cin if parameter FULL_ADDER is high	Randomization under constraints on the A and B to have the maximum, minimum and zero values most of the time		Output Checked against golden model
ALSU_3	Incase of bypass_A and bypass_B, if INPUT_PRIORITY is set to A, the output should have value of A	Randomization under constraints on the bypass_A and bypass_B to off most of the time		Output Checked against golden model
ALSU_4	Incase of red_op_A and red_op_B are set , if INPUT_PRIORITY is set to A, the output should have value of Reduction operation on A	Directed testing		A checker in the testbench to make sure the output is correct
ALSU_5	Incase of invalid cases do occur, the leds should toggle and out is set to zero unless bypass signals are set	Directed testing		A checker in the testbench to make sure the output is correct

4. Test Bench Code:

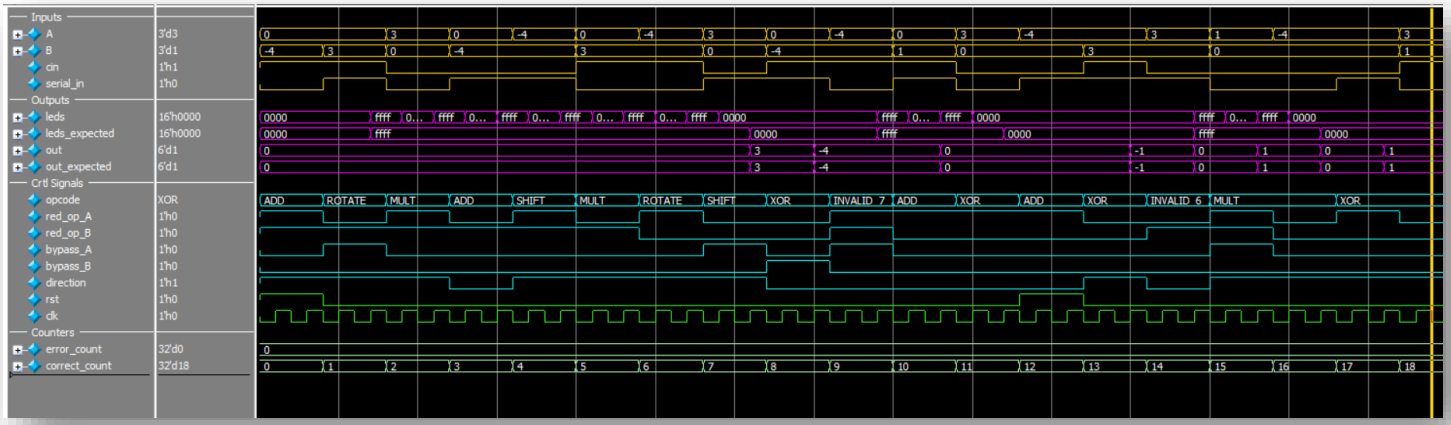
Package:

```
package ALSU_pkg;

    typedef enum logic [2:0]  {OR, XOR, ADD, MULT, SHIFT, ROTATE, INVALID_6, INVALID_7} OPCODE_e;

    parameter INPUT_PRIORITY = "A";
    parameter FULL_ADDER = "ON";
    parameter MAXPOS = 3'b011;
    parameter MAXNEG = 3'b100;
    parameter ZERO = 3'b000;


    class ALSU_inputs;

        rand logic cin;
        rand logic rst;
        rand logic red_op_A;
        rand logic red_op_B;
        rand logic bypass_A;
        rand logic bypass_B;
        rand logic direction;
        rand logic serial_in;
        rand OPCODE_e opcode;
        rand logic signed [2:0] A, B;

        //ALSU_1
        constraint c_rst{
            rst dist{0:/95, 1:/5};
        }

        constraint c_inputs{
            if(opcode != SHIFT || opcode != ROTATE){
                //ALSU_2
                A dist{MAXNEG:/30, ZERO:/30, MAXPOS:/30, [-3:-1]:/10, [1:2]:/10};
                B dist{MAXNEG:/30, ZERO:/30, MAXPOS:/30, [-3:-1]:/10, [1:2]:/10};

                if(((opcode == OR) || (opcode == XOR) && red_op_A)){
                    |A == 1;
                    B == 3'b000;
                }

                if(((opcode == OR) || (opcode == XOR) && red_op_B)){
                    |B == 1;
                    A == 3'b000;
                }
            }
        }

        constraint c_opcode{
            opcode dist{[OR:ROTATE]:/90, [INVALID_6: INVALID_7]:/10};
        }

        constraint c_bypass{
            bypass_A dist{0:/90, 1:/10};
            bypass_B dist{0:/90, 1:/10};
        }
```

Test Bench Code:

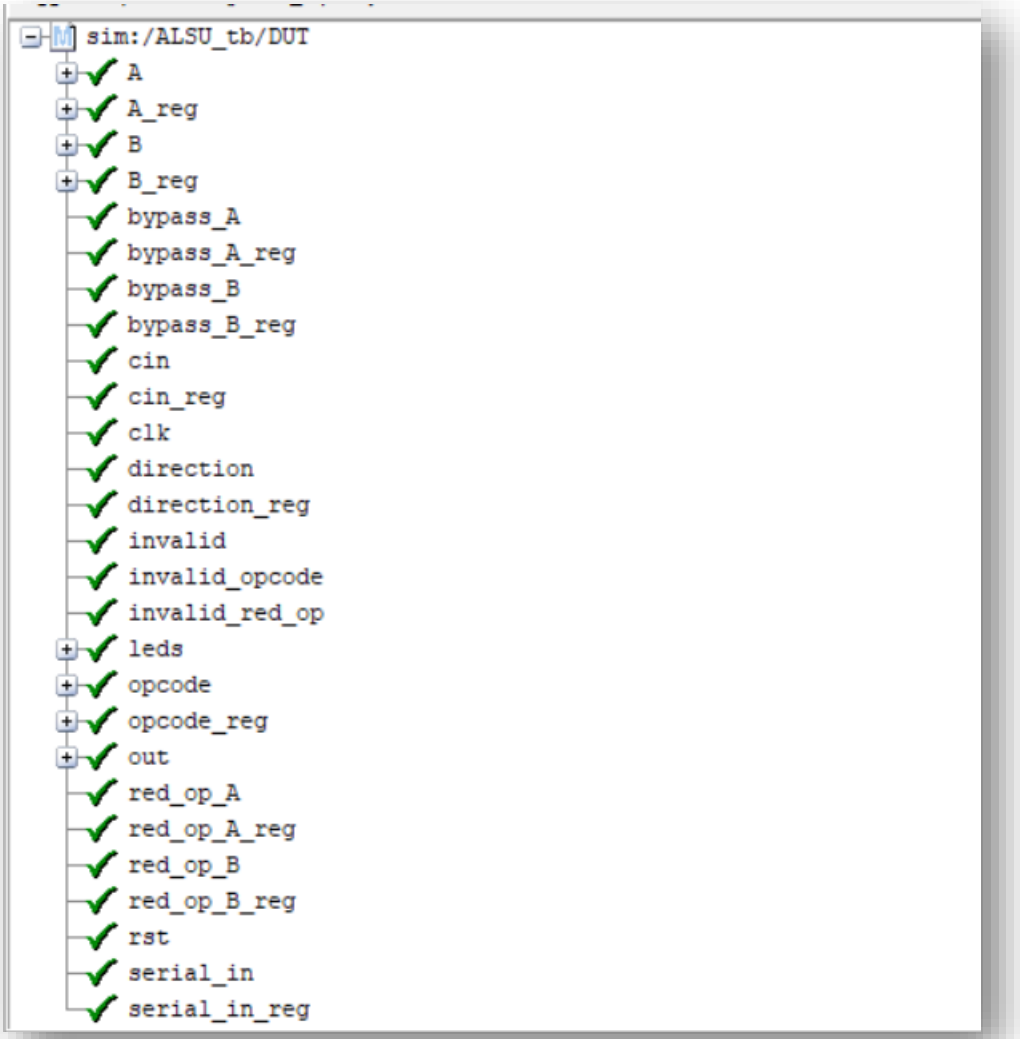
```
import ALSU_pkg::*;

module ALSU_tb ();

    logic clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
    OPCODE_e opcode;
    logic signed [2:0] A, B;
    logic [15:0] leds;
    logic signed [5:0] out;
    //Testbench Variables
    logic [15:0] leds_expected;
    logic signed [5:0] out_expected;

    ALSU_inputs inputs = new();
    int error_count, correct_count;

    ALSU DUT (.*);
    ALSU_golden_2 Golden (.clk(clk), .A(A), .B(B), .cin(cin), .rst(rst), .red_op_A(red_op_A),
                          .red_op_B(red_op_B), .bypass_A(bypass_A), .bypass_B(bypass_B),
                          .direction(direction), .serial_in(serial_in), .opcode(opcode),
                          .leds(leds_expected), .out_DFF(out_expected));

    initial begin
        clk = 0;
        forever
            #1 clk = ~clk;
    end

    initial begin
        //ALSU_1
        assert(inputs.randomize());
        rst        = 1;
        cin        = inputs.cin;
        red_op_A   = inputs.red_op_A;
        red_op_B   = inputs.red_op_B;
        bypass_A   = inputs.bypass_A;
        bypass_B   = inputs.bypass_B;
        direction  = inputs.direction;
        serial_in  = inputs.serial_in;
        opcode     = inputs.opcode;

        A = inputs.A;
        B = inputs.B;
        check_result();

        //ALSU_2, ALSU_3, ALSU_5
        for(int i = 0; i < 200; i++)begin
            assert(inputs.randomize());
            rst        = inputs.rst;
            cin        = inputs.cin;
            red_op_A   = inputs.red_op_A;
            red_op_B   = inputs.red_op_B;
            bypass_A   = inputs.bypass_A;
            bypass_B   = inputs.bypass_B;
            direction  = inputs.direction;
            serial_in  = inputs.serial_in;
            opcode     = inputs.opcode;

            A = inputs.A;
            B = inputs.B;
            check_result();
        end

        //ALSU_4
        A         = 2; B         = 3;
        cin       = 0; serial_in = 0;
        red_op_A  = 1; red_op_B  = 1;
        opcode    = OR; direction = 1;
        bypass_A  = 0; bypass_B  = 0;
        check_result();

        A         = 2; B         = 3;
        cin       = 0; serial_in = 0;
        red_op_A  = 1; red_op_B  = 0;
        opcode    = OR; direction = 1;
        bypass_A  = 0; bypass_B  = 0;
        check_result();

        A         = 2; B         = 3;
        cin       = 0; serial_in = 0;
        red_op_A  = 0; red_op_B  = 1;
        opcode    = OR; direction = 1;
        bypass_A  = 0; bypass_B  = 0;
        check_result();

        A         = -1; B         = 1;
        cin       = 0; serial_in = 0;
        red_op_A  = 1; red_op_B  = 1;
        opcode    = XOR; direction = 1;
        bypass_A  = 0; bypass_B  = 0;
        check_result();



        $display("Total Tests:
 | Correct Tests:
 | Error Test:
", correct_count + error_count, correct_count, error_count);
    task check_result();
        repeat(2)@(negedge clk);
        if((leds !== leds_expected) || (out !== out_expected)) begin
            error_count++;
            $display("Expected Leds:%b, Out:%d| DUT Leds:%b, Out:%d <---- ERROR", leds_expected, out_expected, leds, out);
            //$stop;
        end
        else begin
            $display("Expected Leds:%b, Out:%d| DUT Leds:%b, Out:%d", leds_expected, out_expected, leds, out);
            correct_count++;
        end
    endtask
endmodule
```

5. QuestaSim



6. Code Coverage:

Toggle Coverage:



Statement Coverage:

ALSU.v

```
18 assign invalid_red_op = (red_op_A_reg | red_op_B_reg) & (opcode_reg[1] | opcode_reg[2]);
19 assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
20 assign invalid = invalid_red_op | invalid_opcode;
23 always @(posedge clk or posedge rst) begin
25 cin_reg <= 0;
26 red_op_B_reg <= 0;
27 red_op_A_reg <= 0;
28 bypass_B_reg <= 0;
29 bypass_A_reg <= 0;
30 direction_reg <= 0;
31 serial_in_reg <= 0;
32 opcode_reg <= 0;
33 A_reg <= 0;
34 B_reg <= 0;
36 cin_reg <= cin;
37 red_op_B_reg <= red_op_B;
38 red_op_A_reg <= red_op_A;
39 bypass_B_reg <= bypass_B;
40 bypass_A_reg <= bypass_A;
41 direction_reg <= direction;
42 serial_in_reg <= serial_in;
43 opcode_reg <= opcode;
44 A_reg <= A;
45 B_reg <= B;
50 always @(posedge clk or posedge rst) begin
52 leds <= 0;
55 leds <= ~leds;
57 leds <= 0;
62 always @(posedge clk or posedge rst) begin
64 out <= 0;
68 out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
70 out <= A_reg;
72 out <= B_reg;
74 out <= 0;
79 out <= (INPUT_PRIORITY == "A")? |A_reg: |B_reg;
81 out <= |A_reg;
83 out <= |B_reg;
```

```
85 out <= A_reg | B_reg;
89 out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg;
91 out <= ^A_reg;
93 out <= ^B_reg;
95 out <= A_reg ^ B_reg;
97 3'h2: out <= (FULL_ADDER == "ON")? A_reg + B_reg + cin_reg : A_reg + B_reg ; //Fixed Bug: added FULL_ADDER checker
98 3'h3: out <= A_reg * B_reg;
101 out <= {out[4:0], serial_in_reg};
103 out <= {serial_in_reg, out[5:1]};
107 out <= {out[4:0], out[5]};
109 out <= {out[0], out[5:1]};
E 111 default: out <= 0;
```

Branch Coverage:

```
✓   24 if(rst) begin
✓   35 end else begin
✓   51 if(rst) begin
✓   53 end else begin
✓   54 if (invalid)
✓   56 else
✓   63 if(rst) begin
✓   66 else begin
✓   67 if (bypass_A_reg && bypass_B_reg)
✓   69 else if (bypass_A_reg)
✓   71 else if (bypass_B_reg)
✓   73 else if (invalid)
✓   75 else begin
✓   77 3'h0: begin
✓   78 if (red_op_A_reg && red_op_B_reg)
✓   80 else if (red_op_A_reg)
✓   82 else if (red_op_B_reg)
✓   84 else
✓   87 3'h1: begin
✓   88 if (red_op_A_reg && red_op_B_reg)
✓   90 else if (red_op_A_reg)
✓   92 else if (red_op_B_reg)
✓   94 else
✓   97 3'h2: out <= (FULL_ADDER == "ON")? A_reg + B_reg + cin_reg : A_reg + B_reg ; //Fixed Bug: added FULL_ADDER checker
✓   98 3'h3: out <= A_reg * B_reg;
✓   99 3'h4: begin
✓  100 if (direction_reg)
✓  102 else
✓  105 3'h5: begin
✓  106 if (direction_reg)
✓  108 else
E  111 default: out <= 0;
```

7. Coverage Report

Branch Coverage:

```
================================================================================
=== Instance: /\ALSU_tb#DUT
=== Design Unit: work.ALSU
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        31        31         0   100.00%

================================Branch Details================================
```

Statement Coverage:

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      48        48         0   100.00%

================================Statement Details================================

Statement Coverage for instance /\ALSU_tb#DUT  --
```

Toggle and Total Coverage:

```
Toggle Coverage for instance /\ALSU_tb#DUT  --

                                      Node      1H->0L      0L->1H                          "Coverage"
                                      ----------------------------------------------------------------
                                     A[0-2]           1           1                             100.00
                                   A_reg[2-0]         1           1                             100.00
                                     B[0-2]           1           1                             100.00
                                   B_reg[2-0]         1           1                             100.00
                                    bypass_A          1           1                             100.00
                                bypass_A_reg          1           1                             100.00
                                    bypass_B          1           1                             100.00
                                bypass_B_reg          1           1                             100.00
                                        cin           1           1                             100.00
                                    cin_reg           1           1                             100.00
                                        clk           1           1                             100.00
                                  direction           1           1                             100.00
                              direction_reg           1           1                             100.00
                                    invalid           1           1                             100.00
                               invalid_opcode         1           1                             100.00
                               invalid_red_op         1           1                             100.00
                                   leds[15-0]         1           1                             100.00
                                  opcode[0-2]         1           1                             100.00
                               opcode_reg[2-0]        1           1                             100.00
                                     out[5-0]         1           1                             100.00
                                    red_op_A          1           1                             100.00
                                red_op_A_reg          1           1                             100.00
                                    red_op_B          1           1                             100.00
                                red_op_B_reg          1           1                             100.00
                                         rst          1           1                             100.00
                                   serial_in          1           1                             100.00
                                serial_in_reg         1           1                             100.00

Total Node Count     =         59
Toggled Node Count   =         59
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (118 of 118 bins)


Total Coverage By Instance (filtered view): 100.00%
```